Database Administration (DBA)

Executive Midterm Study Guide & Core Architecture Reference

Course: DBA | Scope: Lectures 1, 2, 3, 4, 5 | Storage & Indexing | Transactions & ACID | High-Yield Exam Prep

01 Executive Overview & Core Concepts (Lecture 1)

Database Administration is a critical **management discipline** focused on orchestrating, securing, and optimizing enterprise data infrastructure. Data is recognized as the ultimate corporate asset; systems and hardware become obsolete, but data persists indefinitely.

> **Core Definition of a DBA**
>
> A **Database Administrator (DBA)** is the information technician responsible for ensuring the operational environment and directing the infrastructure for an organization's data.

1.1 What Makes a Good DBA? (Avoiding Anti-Patterns)

- **Jack of All Trades:** The DBA must bridge software development, operating systems, hardware configuration, network security, and business domain logic.
- **The Curmudgeon in the Corner (Prima Donna Anti-Pattern):** A negative persona to avoid at all costs—an unapproachable technician who reflexively says "No" to developers and business requests, creating an enterprise bottleneck.
- **A Good DBA:** Collaborates with developers early in the SDLC, proactively designs for scalability, documents procedures, and adopts automated monitoring rather than firefighting.

1.2 Database 101: Database vs. DBMS

Concept	Definition	Examples / Scope
Database	An organized, structured store of persistent data models representing entities, relationships, and constraints.	CustomerDB, Sales_2026, MedicalRecords
DBMS	The sophisticated software engine that creates, manages, optimizes, secures, and controls access to databases.	Oracle Database, IBM DB2, Microsoft SQL Server, PostgreSQL, MySQL

1.3 Administrative Roles Comparison

Role	Primary Responsibility	Key Focus Area
Data Administrator (DA / CDO)	Business data governance, metadata policies, business glossaries, conceptual/logical data modeling.	Business meaning, data ownership, regulatory data classification.
System Administrator (SA)	Operating system, hardware servers, storage RAID arrays, network routing, OS-level security.	Physical server health, OS kernel patches, memory/disk allocation.
Database Administrator (DBA)	DBMS software installation, physical database realization, SQL query optimization, disaster recovery, ACID transaction management.	DBMS instance configuration, index tuning, backup/recovery, high availability.

1.4 Specialized Types of DBAs

- **System DBA:** Configures the DBMS software engine, tunes DBMS memory pools/buffer caches, applies vendor software patches, and executes version migrations.
- **Application DBA:** Embedded with software engineering teams, specializing in query optimization, table schemas, and data access logic for specific business applications.
- **Database Architect & Analyst:** Blueprints enterprise-wide data flows, integration pipelines, and long-term infrastructure roadmaps.
- **Data Modeler:** Translates business requirements into normalized Conceptual and Logical Entity-Relationship Diagrams (ERDs).
- **Data Warehouse Administrator (DWA):** Manages OLAP systems, star/snowflake schemas, ETL loading pipelines, and massive analytical data stores.
- **Task-Oriented DBA:** Focuses exclusively on specialized enterprise domains such as Disaster Recovery, Security/Compliance Auditing, or Performance Diagnostics.
- **Procedural DBA:** Manages procedural logic inside the database engine (Stored Procedures, Triggers, User-Defined Functions - UDFs) to ensure they do not cause deadlocks or drain server CPU.
- **Cloud DBA:** Operates in Managed Cloud Database environments (DBaaS like AWS RDS, Azure SQL, Google Cloud SQL). Low-level hardware maintenance is offloaded, shifting the DBA's focus to cloud cost optimization, autoscaling, and data security.

1.5 DBA Reporting Structures & Staffing Rules of Thumb

- **Centralized (Infrastructure/Operations):** Fosters standardized corporate policies, uniform backup/DR protocols, and independent architectural oversight.
- **Decentralized (Application Teams):** Enhances agility and application responsiveness, but risks creating isolated data silos and non-standard schemas.
- **Staffing Determinants:** DBA headcount is determined by instance count, total data volume, transaction throughput (OLTP vs DSS), 24/7 SLA uptime demands, and the maturity of automated monitoring tools.

02 Physical Database Design, Storage & Indexing (Lectures 2 & 3)

Physical database design translates the logical data model into physical storage structures within the target DBMS, optimizing for performance, storage efficiency, and data integrity.

2.1 Translation from Logical to Physical

Logical Concept	Physical Implementation	Design Decisions & Rules of Thumb (ROTs)
Entity	Table	Choose table space, partition key, and row format.
Attribute	Column	Assign exact data type (INT, VARCHAR, TIMESTAMP), NULL / NOT NULL rules, DEFAULT values.
Primary Identifier	Primary Key Constraint	Backed by a unique index (frequently clustered).
Relationship	Foreign Key Constraint	Enforces referential integrity (ON DELETE CASCADE, SET NULL, RESTRICT).

> **Physical ROTs: Column Ordering & Sizing**
>
> Place fixed-length, frequently searched columns at the beginning of the row layout (allowing direct offset calculation by the CPU), and variable-length columns (VARCHAR, LOBs) at the end. Always avoid arbitrary over-allocation of column lengths.

2.2 Storage Calculation & Free Space Management

- **Row Length:** Sum of column lengths + row overhead header bytes + null indicator bytes.
- **Page/Block Capacity:** Data pages (typically 4KB, 8KB, or 16KB) contain a page header, directory slots, and stored rows.
- **PCTFREE:** The percentage of each page reserved during initial load to allow existing rows to expand (e.g. updating a short VARCHAR to a longer string) without causing *Row Migration* or *Row Chaining*.
- **FREEPAGE:** Specifies that an entire empty page is left every *N* pages to provide space for localized bulk inserts to maintain physical clustering.

2.3 B-Tree vs. Bitmap Indexes (Comprehensive Comparison)

Attribute	B-Tree Index (B+ Tree)	Bitmap Index
Structure	Multi-level balanced tree: Root page, Intermediate branch pages, and sorted Leaf pages containing keys and RowIDs.	Array of bit vectors (0s and 1s) for each distinct column value; each bit position corresponds to a physical row number.

Best For	High cardinality columns (Primary keys, Social Security Numbers, timestamps). Point queries and range searches.	Low cardinality columns (Gender, Marital Status, Country, Boolean flags) in read-heavy Data Warehouses (OLAP).
Query Operations	Exact match (`=`), Range scans (`BETWEEN`, `>`, `<`), Sorting (`ORDER BY`).	Boolean bitwise operations (AND, OR, NOT) executed at lightning speed by the CPU across multiple indexed columns.
Concurrency / OLTP	High concurrency; row-level locking supported on leaf pages.	**Poor for OLTP:** Updating a single row locks an entire bitmap segment, blocking concurrent transactions.

2.4 Clustered vs. Non-Clustered Indexes

- **Clustered Index:** The physical storage order of rows in the table matches the logical order of the index key. Because physical table rows can only be arranged on disk in one physical sequence, a table can have **only ONE clustered index**.
- **Non-Clustered Index:** A separate index structure containing sorted keys and pointers (RowIDs or clustered index keys) back to the actual data rows. A table can have many non-clustered indexes.
- **Covering Index (Index-Only Scan):** When an index contains all columns requested by a SELECT and WHERE clause, the query engine satisfies the request entirely from the index without reading base table pages.

2.5 Partitioning & Denormalization

- **Partitioning:** Divides large tables into smaller physical segments based on a key (Range, Hash, or List). Enables *partition pruning* (skipping unneeded partitions during queries) and fast maintenance (dropping an old year partition instantly).
- **Denormalization:** The deliberate introduction of controlled redundancy or grouping of tables into a normalized schema to eliminate costly multi-table JOINs in read-heavy workloads.
  - *Types:* Pre-joined tables, derived/calculated summary columns, duplicate redundant attributes.
  - *Costs & Risks:* Increased storage, update anomalies, and data integrity degradation if updates fail to sync.

2.6 Temporal Data Management

- **Business Time (Valid Time / Application Time):** The real-world time period during which a fact was true (e.g., insurance coverage valid from Jan 1 to Dec 31). Managed by application logic.
- **System Time (Transaction Time):** The time period during which the record was stored and active in the database system. Managed automatically by the DBMS engine.
- **Bi-Temporal Tables:** Support both Business Time and System Time simultaneously, providing an indisputable audit trail of real-world history and system modifications.

03 Application Development, SQL, Transactions & Locking (Lectures 4 & 5)

3.1 SQL Architecture & Embedded SQL

- **Declarative Language:** In SQL, the developer declares *WHAT* data is required, not *HOW* to retrieve it. The DBMS Cost-Based Optimizer (CBO) determines the optimal access path.
- **Set-at-a-Time:** Relational operations execute against entire sets of rows simultaneously, unlike 3GL procedural languages that loop record-by-record.
- **Embedded SQL & Host Variables:** Embedded SQL places SQL queries inside host languages (C, COBOL). Host variables (prefixed with `:`) exchange parameters between the program and the DBMS.
- **Cursors:** A procedural pointer mechanism allowing a 3GL program to iterate through a multi-row SQL result set row-by-row (`DECLARE -> OPEN -> FETCH -> CLOSE`).

3.2 Middleware, Object-Relational Mapping & Impedance Mismatch

- **Middleware APIs:** ODBC, JDBC, and ADO.NET provide database-neutral APIs. The *Driver Manager* routes application calls to the appropriate vendor-specific driver.
- **Object-Relational Impedance Mismatch:** The fundamental architectural tension between the Object-Oriented paradigm (classes, polymorphism, object references) and the Relational model (tables, foreign keys, relational algebra).
- **ORM Frameworks (Hibernate, Entity Framework):** Automate mapping between objects and tables. DBAs must monitor ORMs to prevent severe performance anti-patterns, such as the **N+1 SELECT problem**.

3.3 ACID Properties of Transactions

Property	Name	Guarantee & Description
A	Atomicity	The "All-or-Nothing" principle: either every operation within the transaction succeeds and commits, or all changes are rolled back completely.
C	Consistency	Transactions move the database from one valid state satisfying all schema constraints, checks, and foreign keys to another valid state.
I	Isolation	Concurrently executing transactions do not interfere with each other; uncommitted changes are shielded from other concurrent sessions.
D	Durability	Once committed, updates are permanently written to non-volatile storage (via write-ahead transaction logs) and survive system crashes.

3.4 Concurrency Control: Lock Types & Deadlocks

- **Shared Lock (S Lock):** Acquired for reading data. Multiple transactions can hold S locks on the same resource concurrently.

- **Exclusive Lock (X Lock):** Required for writing or updating data. Grants exclusive access; no other transaction can hold S or X locks on that resource.
- **Update Lock (U Lock):** A hybrid lock acquired during read phases before an update to prevent conversion deadlocks.
- **Intent Locks (IS, IX, SIX):** Placed at higher levels of the locking hierarchy (table/page) to signal that fine-grained locks exist below.
- **Deadlocks:** Circular wait conditions (Transaction A holds Resource 1 and waits for Resource 2; Transaction B holds Resource 2 and waits for Resource 1). The DBMS maintains a *Wait-For Graph (WFG)*, detects cycles, and aborts a "victim" transaction.
- **Lock Escalation:** When a session acquires too many individual row locks, the engine automatically converts them into a single table lock to save memory, severely reducing concurrency.

3.5 ANSI SQL Isolation Levels & Concurrency Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Default in Most DBMS
Read Uncommitted	Permitted	Permitted	Permitted	No
Read Committed	**Prevented**	Permitted	Permitted	**Yes (Standard Default)**
Repeatable Read	**Prevented**	**Prevented**	Permitted	No
Serializable	**Prevented**	**Prevented**	**Prevented**	No (Strict / Slowest)

Core Programming Guidelines for DBAs & Developers

- Keep transactions as short as possible; never wait for user input inside an active transaction.
- Access database tables and rows in the **exact same sequential order** across all applications to eliminate deadlocks.
- Use parameterized queries to enable execution plan caching and prevent SQL injection.
- In large batch jobs, commit in intermediate chunks (e.g. every 5,000 rows) to avoid transaction log exhaustion and lock escalation.